

Estudio de viabilidad- Generacion de reportes para el contador

1. Descripción del componente

El componente de generación de reportes debía resolver la necesidad del contador de consultar información consolidada sobre la actividad tributaria del estudio contable. Su objetivo principal es transformar los datos operativos del sistema en información útil para la toma de decisiones y el seguimiento administrativo.

Este componente consume datos provenientes de la base de datos relacional Neon PostgreSQL, accedida mediante Prisma ORM. Entre los datos utilizados se encuentran clientes, liquidaciones, comprobantes, impuestos, entidades tributarias, estados de pago, fechas de vencimiento e importes. También puede utilizar información de relación entre clientes, impuestos y entidades tributarias para construir reportes más específicos.

Como resultado, el componente genera listados, resúmenes o reportes consultables desde el panel del contador. En una primera etapa, estos reportes podrían visualizarse en pantalla mediante filtros por cliente, período, estado, impuesto o entidad tributaria. En una etapa posterior, podrían exportarse en formatos como PDF o Excel.

Dentro de la arquitectura, el componente se integra como un módulo funcional del monolito modular desarrollado en Next.js. La interfaz se implementaría mediante páginas y componentes React dentro del dashboard del contador, mientras que la obtención y procesamiento de datos se realizaría del lado del servidor mediante Server Components, Server Actions o funciones auxiliares. La persistencia se mantiene centralizada en Prisma ORM y Neon PostgreSQL, siguiendo la misma estrategia utilizada por el resto del sistema.

En términos de criticidad, este componente no es indispensable para que el sistema ejecute sus operaciones principales, como registrar clientes, cargar liquidaciones, subir comprobantes o enviar recordatorios. Sin embargo, tiene una importancia alta para la gestión administrativa, ya que mejora la visibilidad del estado tributario del estudio y permite detectar vencimientos, pagos pendientes y tendencias operativas. Por lo tanto, se considera un componente de criticidad media-alta: no bloquea el funcionamiento básico del sistema, pero aporta valor significativo para el control y seguimiento del negocio.

2. Alternativas exploradas

Alternativa 1 (A1): Generación dinámica de reportes en la aplicación

Esta alternativa consiste en generar los reportes en el momento en que el contador los solicita desde el panel administrativo. El sistema consulta la base de datos mediante Prisma ORM, aplica los filtros seleccionados, por ejemplo cliente, período, estado, impuesto o entidad tributaria, y muestra el resultado en pantalla.

Resolvería el problema permitiendo al contador consultar información actualizada en tiempo real, sin necesidad de almacenar reportes previamente generados. Cada vez que se accede al reporte, los datos se obtienen directamente desde la base de datos Neon PostgreSQL.

Se consideró una opción válida porque aprovecha la arquitectura actual del sistema, basada en Next.js, Prisma y PostgreSQL. No requiere servicios externos adicionales ni tareas programadas nuevas, y resulta adecuada para el volumen esperado del proyecto.

Alternativa 2 (A2): Generación programada mediante cron job

Esta alternativa consiste en generar o actualizar reportes automáticamente en horarios definidos, por ejemplo una vez por día. Un cron job ejecutaría un proceso que consulta la base de datos, calcula los indicadores o resúmenes necesarios y almacena los resultados para que luego el contador los consulte rápidamente.

Resolvería el problema reduciendo el tiempo de carga de los reportes, ya que los datos estarían precalculados. Sería útil si los reportes fueran costosos de generar o si el volumen de información creciera considerablemente.

Se consideró una opción válida porque el sistema ya utiliza Vercel Cron para procesos automáticos, como el control de vencimientos. Sin embargo, presenta una limitación importante: en planes gratuitos existe una restricción en la cantidad o frecuencia de cron jobs disponibles. Además, los reportes no estarían necesariamente actualizados en tiempo real, sino según la última ejecución programada y la generación programada solo resultaría conveniente ante volúmenes de datos significativamente mayores o reportes con cálculos complejos, situación que no se espera en el contexto actual del sistema

Alternativa 3 (A3): Herramienta externa de reportes o BI

Esta alternativa consiste en utilizar una herramienta externa especializada en reportes o visualización de datos, como Metabase, Looker Studio u otra solución de Business Intelligence. Estas herramientas se conectan a la base de datos o a una fuente intermedia y permiten construir paneles, gráficos y reportes sin desarrollar toda la funcionalidad desde cero.

Resolvería el problema ofreciendo capacidades avanzadas de visualización, filtros, gráficos y exportación. Podría ser útil para obtener reportes más complejos o tableros administrativos.

Se consideró una opción válida porque reduce el desarrollo propio de visualizaciones y aprovecha herramientas ya diseñadas para análisis de datos. Sin embargo, agrega dependencia de un servicio externo, configuración adicional, posibles costos y mayores cuidados de seguridad, ya que se estaría exponiendo información sensible de clientes y liquidaciones a una herramienta externa.

3. Criterios de evaluación

Factibilidad técnica

A1 - Generación dinámica:

Alta. Es compatible con la arquitectura actual basada en Next.js, Prisma y PostgreSQL. El equipo ya utiliza estas tecnologías en otras funcionalidades del sistema.

A2 - Cron job:

Media. El equipo ya implementó un cron job para recordatorios de vencimientos, por lo que la tecnología es conocida. Sin embargo, requeriría definir qué datos se precaculan, dónde se almacenan y cómo se actualizan.

A3 - Herramienta BI:

Media-Baja. Existen herramientas que facilitan la generación de reportes, pero el equipo debería aprender a configurarlas, conectarlas de forma segura a la base de datos y administrar permisos.

Factibilidad temporal

A1 - Generación dinámica:

Alta. Es la alternativa más rápida para una primera versión, ya que consiste en crear una pantalla de reportes con consultas y filtros sobre datos existentes.

A2 - Cron job:

Media. Requiere más tiempo que la generación dinámica, porque además de construir los reportes hay que implementar el proceso programado, almacenar resultados y contemplar actualizaciones.

A3 - Herramienta BI:

Media-Baja. Puede acelerar la creación visual de dashboards, pero requiere tiempo de configuración, conexión con la base, permisos, despliegue o integración externa.

Integración con la arquitectura

A1 - Generación dinámica:

Alta. Encaja naturalmente con el monolito modular actual. Puede implementarse como un módulo más dentro del dashboard del contador, usando Server Components, Server Actions y consultas a Prisma.

A2 - Cron job:

Media. Encaja con la arquitectura porque el sistema ya utiliza Vercel Cron, pero agregaría otro proceso automático y posiblemente nuevas tablas o campos para almacenar reportes precaculados.

A3 - Herramienta BI:

Media-Baja. Se integra de forma más externa a la arquitectura. Puede conectarse a PostgreSQL, pero no forma parte directa del flujo de Next.js y podría requerir accesos adicionales a la base de datos.

Complejidad de mantenimiento

A1 - Generación dinámica:

Baja-Media. El mantenimiento queda dentro del mismo código del sistema. Las consultas y pantallas se versionan junto con el resto de la aplicación.

A2 - Cron job:

Media-Alta. Requiere mantener lógica de generación, horarios de ejecución, almacenamiento de resultados y posibles reprocesamientos si cambian los datos.

A3 - Herramienta BI:

Media. Reduce el mantenimiento de gráficos y dashboards, pero agrega mantenimiento de configuración externa, credenciales, permisos y conectores.

Escalabilidad futura

A1 - Generación dinámica:

Media. Es adecuada para un volumen bajo o moderado de datos. Si el volumen crece mucho, las consultas podrían volverse pesadas y requerir optimización, paginación o precálculo.

A2 - Cron job:

Alta. Puede escalar mejor para reportes pesados, porque los datos se calculan previamente y el usuario consulta resultados ya procesados.

A3 - Herramienta BI:

Alta. Muchas herramientas BI están pensadas para manejar dashboards y análisis de datos más complejos. Sin embargo, la escalabilidad depende del proveedor, la configuración y el plan utilizado.

Costo / dependencia externa

A1 - Generación dinámica:

Bajo. No requiere proveedores adicionales. Usa las tecnologías ya incorporadas en el proyecto.

A2 - Cron job:

Medio. No necesariamente requiere un proveedor nuevo, pero puede tener limitaciones según el plan de Vercel. En la versión gratuita que utilizamos, sumar cron jobs o aumentar frecuencia puede implicar restricciones.

A3 - Herramienta BI:

Medio-Alto. Puede generar dependencia de una herramienta externa y posibles costos según la plataforma elegida. También requiere exponer o conectar datos sensibles del sistema con un servicio adicional.

4. Tabla comparativa

Criterio	A1- Dinámica	A2- Cron	A3- BI externa
Factibilidad técnica	Alta	Media	Media-Baja
Factibilidad temporal	Alta	Media	Media-Baja
Integración arquitectura	Alta	Media	Media-Baja
Complejidad de mantenimiento	Baja-Media	Media-Alta	Media

Escalabilidad futura	Media	Alta	Alta
Costo/dependencia externa	Bajo	Medio	Medio-Alto

5. Conclusiones

A partir del análisis realizado, el grupo selecciona la alternativa de generación dinámica de reportes dentro de la aplicación. Esta solución consiste en incorporar un módulo de reportes en el dashboard del contador, permitiendo consultar información mediante filtros por cliente, período fiscal, estado de liquidación, impuesto y entidad tributaria.

La alternativa fue elegida por presentar la mayor factibilidad técnica y temporal, integrarse de forma natural con la arquitectura actual y no requerir dependencias externas ni mecanismos adicionales de procesamiento. La implementación aprovecharía la infraestructura ya utilizada por el sistema, compuesta por Next.js, Prisma ORM y Neon PostgreSQL, permitiendo generar los reportes directamente a partir de los datos operativos existentes.

La funcionalidad no fue desarrollada durante el proyecto debido a las restricciones de tiempo y a la necesidad de priorizar componentes considerados esenciales para el funcionamiento del sistema, como la gestión de clientes, liquidaciones, comprobantes, entidades tributarias, impuestos, notificaciones y vencimientos. Además, sería necesario profundizar el relevamiento de requerimientos para determinar qué reportes e indicadores aportan mayor valor al contador.

Desde el punto de vista arquitectónico, esta alternativa se integraría como un nuevo módulo funcional dentro del monolito modular existente. Se agregaría una sección “Reportes” en el dashboard del contador. La interfaz estaría implementada con componentes React, mientras que la obtención de datos se realizaría desde el servidor mediante Server Components, Server Actions o funciones auxiliares. Las consultas se resolverían mediante Prisma ORM sobre la base de datos Neon PostgreSQL.

Como trabajo futuro, podrían incorporarse funcionalidades adicionales como exportación a PDF o Excel, visualizaciones gráficas y reportes más avanzados. En caso de que el volumen de datos o la complejidad de las consultas aumenten significativamente, podrían reevaluarse alternativas basadas en reportes precalculados o herramientas especializadas de Business Intelligence.

La decisión prioriza bajo costo, integración simple y aprovechamiento de la arquitectura existente, dejando la generación programada o el uso de herramientas BI como alternativas futuras en caso de crecimiento del volumen de datos o necesidad de reportes más complejos.